Simulating GAME Agents in a Conversation

Testing Agent Designs Through Conversation Simulation

Before we write a single line of code for our agent, we should test whether our GAME design is actually feasible. One powerful technique is to simulate the agent's decision-making process through conversation with an LLM in a chat interface (e.g., ChatGPT). This approach helps us identify potential problems with our design early, when they're easiest to fix. Let's explore how to conduct these simulations effectively.

Why Simulate First?

Think of agent simulation like a dress rehearsal for a play. Before investing in costumes and sets, you want to make sure the script makes sense and the actors can perform their roles effectively. Similarly, before implementing an agent, we want to verify that:

1. The goals are achievable with the planned actions
2. The memory requirements are reasonable
3. The actions available are sufficient to solve the problem
4. The agent can make appropriate decisions with the available information

Setting Up Your Simulation

When starting a conversation with an LLM to simulate your agent, begin by establishing the framework. We can do this with a simple prompt in a chat interface. The prompt should clearly outline the agent's goals, actions, and the simulation process. Here's a template you can use:

```
I'd like to simulate an AI agent that I'm designing. The agent will be built using these components:

Goals: [list your goals]
Actions: [list available actions]

At each step, your output must be an action to take.

Stop and wait and I will type in the result of
the action as my next message.

Ask me for the first task to perform.
```

For a *Proactive Coder* agent, you might use the following prompt to kick-off a simulation in ChatGPT:

```
I'd like to simulate an AI agent that I'm designing. The agent will be built using these components:

Goals:
* Find potential code enhancements
* Ensure changes are small and self-contained
* Get user approval before making changes
* Maintain existing interfaces

Actions available:
* list_project_files()
* read_project_file(filename)
* ask_user_approval(proposal)
* edit_project_file(filename, changes)

At each step, your output must be an action to take.

Stop and wait and I will type in the result of
the action as my next message.

Ask me for the first task to perform.
```

Take a moment to open up ChatGPT and try out this prompt. You can use the same prompt in any chat interface that supports LLMs. What worked? What didn't?

Learning Through Agent Simulation

Understanding Agent Reasoning

When you begin simulating your agent's behavior, you're essentially conducting a series of experiments to understand how well it can reason with the tools and goals you've provided. Start by presenting a simple scenario – perhaps a small Python project with just a few files. Watch how the agent approaches the task. Does it immediately jump to reading files, or does it first list the available files to get an overview? These initial decisions reveal a lot about whether your goals and actions enable systematic problem-solving.

As you observe the agent's decisions, you'll notice that the way you present information significantly impacts its reasoning. For instance, when you return the results of list_project_files(), you might first try returning just the filenames:

```
["main.py", "utils.py", "data_processor.py"]
```

Then experiment with providing more context:

```
{
    "files": ["main.py", "utils.py", "data_processor.py"],
    "total_files": 3,
    "directory": "/project"
}
```

You might discover that the additional metadata helps the agent make more informed decisions about which files to examine next. This kind of experimentation with result formats helps you understand how much context your agent needs to reason effectively.

Evolving Your Tools and Goals

The simulation process often reveals that your initial tool descriptions aren't as clear as you thought. For example, you might start with a simple description for read_project_file():

```
read_project_file(filename) -> Returns the content of the specified file
```

Through simulation, you might find the agent using it incorrectly, leading you to enhance the description:

```
read_project_file(filename) -> Returns the content of a Python file from the project directory.
The filename should be one previously returned by list_project_files().
```

Similarly, your goals might evolve. You might start with "Find potential code enhancements" but discover through simulation that the agent needs more specific guidance. This might lead you to refine the goal to "Identify opportunities to improve error handling and input validation in functions."

Understanding Memory Through Chat

One of the most enlightening aspects of simulation is realizing that the chat format naturally mimics the list-based memory system we use in our agent loop memory. Each exchange between you and the LLM represents an iteration of the agent loop and a new memory entry – the agent's actions and the environment's responses accumulate just as they would in our implemented memory system. This helps you understand how much history the agent can accumulate and still maintain context and make good decisions.

Learning from Failures

Introducing controlled chaos into your simulation provides valuable insights. Try returning error messages instead of successful results:

```
{"error": "FileNotFoundError: main.py does not exist"}
```

Or return malformed data:

```
{"content": "def broken_func(): pass"}
```

Watch how the agent handles these situations. Does it try alternative approaches? Does it give up too easily? Does it maintain its goal focus despite errors? These observations help you design better error handling and recovery strategies.

Preventing Runaway Agents

The simulation environment provides a safe space to test termination conditions. You can experiment with different criteria for when the agent should conclude its task. Perhaps it should stop after examining a certain number of files, or after making a specific number of improvement suggestions. The chat format lets you quickly try different approaches without worrying about infinite loops or resource consumption.

Rapid Iteration and Improvement

The true power of simulation lies in its speed. You can test dozens of scenarios in the time it would take to implement a single feature. Want to see how the agent handles a project with 100 files? Just tell it that's what list_project_files() returned. Curious about how it would handle deeply nested function calls? Paste in some complex code and see how it analyzes it.

Learning from the Agent

At the end of your simulation sessions, ask the agent to reflect on its experience. What tools did it wish it had? Were any instructions unclear? Which goals were too vague? The LLM can often provide surprisingly insightful suggestions about how to improve your GAME design.

For example, the agent might suggest: "The ask_user_approval() action would be more effective if it could include code snippets showing the proposed changes. This would help users make more informed decisions about the suggested improvements."

Building Your Example Library

As you conduct these simulations, you're building a valuable library of examples. When you see the agent make a particularly good decision, save that exchange. When it makes a poor choice, save that too. These examples become invaluable when you move to implementation – they can be used to craft better prompts and test cases.

Keep a record of exchanges like this:

Good Example:

```
Agent: "Before modifying utils.py, I should read its contents to understand the current error handling patterns."
Action: read_project_file("utils.py")
Result: [file contents]
Agent: "I notice these functions lack input validation. I'll propose focused improvements for each function."
```

Poor Example:

```
Agent: "I'll start editing all the files to add error handling."
Action: edit_project_file("utils.py", {...})
[Missing analysis and user approval steps]
```

These examples help you understand what patterns to encourage or discourage in your implemented agent.

Through this iterative process of simulation, observation, and refinement, you develop a deep understanding of how your agent will behave in the real world. This understanding is invaluable when you move to implementation, helping you build agents that are more robust, more capable, and better aligned with your goals.

Remember, the time spent in simulation is an investment that pays off in better design decisions and fewer implementation surprises. When you finally start coding, you're not just hoping your design will work – you've already seen it work in hundreds of scenarios.